

Database normalization

Introduction

This article introduces database normalization, a key concept in database design. When a large number of data records are stored in a relational database, either in a single table or across multiple tables, normalization helps store the information efficiently and optimally.

Benefits of normalization

There are three main benefits to using normalization:

1. Minimize storage usage
2. Maintain data integrity
3. Reduce duplicate data

Normal forms overview

The normalization process produces a compact, efficient, and optimal relational database.

Typically, in database design, there are 3 **normal** forms:

1. First **normal** form (1NF)
2. Second **normal** form (2NF)
3. Third **normal** form (3NF)

A variant of 3NF exists, also known as **Boyce-Codd normal** form. It is stronger than 3NF.

There are 3 more advanced **normal** forms: the fourth, the fifth, and the sixth **normal** forms.

These are not discussed in this article.

For each **normal** form, there are well-defined criteria that the entities and relationships in the database must qualify for the database to be of that **normal** form. For example, if the entities and relationships of a database meet the criteria specified for 1NF, then the database is said to be in 1NF. Similarly, if a database meets the conditions defined for 2NF, then it is said to be in 2NF. The same holds for 3NF.

The criteria get progressively stricter as we go from 1NF to 2NF to 3NF.

If a database is 2NF, then it conforms to 1NF and 2NF, as 2NF includes and builds on 1NF.

Similarly, if a database is 3NF, then it conforms to 1NF, 2NF, and 3NF.

Normalization is the process of analyzing a database and redesigning its tables and columns to conform to the highest possible **normal** form.

If a database is not normalized, it can result in:

1. Data duplication
2. Wasted storage space
3. Insertion anomalies
4. Update anomalies
5. Deletion anomalies

Example database

In this article, a database consisting of tables that store patient, doctor, and related information is used to illustrate normalization concepts.

First normal form (1NF)

For a database to be 1NF:

The data stored in each column of all tables in the database must be **atomic**. **Atomic** means multiple data values should not be stored in a column.

For example, in Table 1-1 below, the table has a **PHONE_NUMBER** column. Only one phone number should be stored for each row. Two or more phone numbers should not be stored in this column. Storing only one phone number makes it **atomic**.

Table 1-1 below represents the database table PATIENT. It has 5 data rows and 4 data columns.

TABLE: PATIENT			
PATIENT_ID	FIRST_NAME	LAST_NAME	PHONE_NUMBER
1	John	Doe	(650) 906 4312
2	Jane	Doe	(408) 735 3123
3	Bob	Taylor	(408) 532 2134 (650) 415 5122
4	James	Lamb	(310) 123 2345
5	Liza	Boyle	(872) 965 5010

Table 1-1: Example of 1NF violation

In the table above, the third row has two phone numbers stored in the **PHONE_NUMBER** column. The first number is the patient's home phone number, and the second is the patient's alternate phone number. Storing both numbers in the same row may appear logical and convenient; however, it can cause update and deletion anomalies that result in data loss.

For example, an update statement below that updates the home phone number for this patient record with a new phone number may overwrite the column, losing the second phone number that existed. This is termed an update anomaly.

```
UPDATE TABLE PATIENT SET PHONE_NUMBER = '(708) 986-5312' WHERE PATIENT_ID = 3
```

This anomaly occurs because the phone number data in row 3 is not atomic and has two values. Therefore, it is not a good design choice to store multiple phone numbers in the **PHONE_NUMBER** column for one patient record. The table violates 1NF as the **PHONE_NUMBER** column has non-atomic values.

To correct the above design issue, create a new table, **ALTERNATE_PHONE_NUMBER**, to store an alternate phone number for patients.

TABLE: ALTERNATE_PHONE_NUMBER	
PATIENT_ID	PHONE_NUMBER
3	(650) 415 5122

Table 1-2: Alternate phone number table

Now, the database has two tables:

1. **PATIENT** stores only 1 phone number for each patient.
2. **ALTERNATE_PHONE_NUMBER** stores an alternate phone number for the patient if one exists.

After the design change, the two tables look like:

TABLE: PATIENT			
PATIENT_ID	FIRST_NAME	LAST_NAME	PHONE_NUMBER
1	John	Doe	(650) 906 4312
2	Jane	Doe	(408) 735 3123
3	Bob	Taylor	(408) 532 2134
4	James	Lamb	(310) 123 2345
5	Liza	Boyle	(872) 965 5010

Table 1-3: PATIENT table stores only 1 phone number

TABLE: ALTERNATE_PHONE_NUMBER	
PATIENT_ID	PHONE_NUMBER
3	(650) 415 5122

Table 1-4: **ALTERNATE_PHONE_NUMBER** stores alternate phone number

With the change above, the database is now in 1NF.

Second normal form (2NF)

For a database to be 2NF:

1. It must be 1NF
2. All non-key columns in each table must be completely dependent on the primary key column. If the primary key is composite, each non-key column must fully depend on the composite primary key. If a table has a composite primary key, and one of the non-key columns does not fully depend on the composite primary key, the table is not in 2NF.

Consider a table that has a composite primary key column and other columns. Table 2-1 below, **PATIENT_DOCTOR**, contains a composite primary key comprising **PATIENT_ID** and **DOCTOR_ID**. The table also has a **DOCTOR_NAME** column. In this table, the **DOCTOR_NAME** column depends only on **DOCTOR_ID** and not on **PATIENT_ID**. Therefore, the table is not in 2NF.

TABLE: PATIENT_DOCTOR		
PATIENT_ID	DOCTOR_ID	DOCTOR_NAME
1	2	Dr John Reid
2	1	Dr Taylor Swift
3	3	Dr Justin Bieber

Table 2-1: **PATIENT_DOCTOR** table storing patients and doctors treating them

There is a problem with this table design. The table has a composite primary key that comprises the **PATIENT_ID** and **DOCTOR_ID** key columns. However, the non-key column

DOCTOR_NAME depends only on the **DOCTOR_ID** key column. It does not depend on the **PATIENT_ID** column at all.

As a result, an update statement below that updates the **DOCTOR_ID** to 3 for **PATIENT_ID** 2 does not update the **DOCTOR_NAME** column. It updates only the **DOCTOR_ID**, resulting in inconsistent information about **DOCTOR_ID** 3.

```
UPDATE TABLE PATIENT_DOCTOR SET DOCTOR_ID = 3 WHERE PATIENT_ID = 2
```

After the update, the table is in Table 2-2. In this table, even though **DOCTOR_ID** is 3 in the second and third rows, the **DOCTOR_NAME** does not match.

TABLE: PATIENT_DOCTOR		
PATIENT_ID	DOCTOR_ID	DOCTOR_NAME
1	2	Dr John Reid
2	3	Dr Taylor Swift
3	3	Dr Justin Bieber

Table 2-2: Update **DOCTOR_ID** leaves **DOCTOR_NAME** column inconsistent

A design change to bring this database up to 2NF would be to:

1. Remove the **DOCTOR_NAME** column from the **PATIENT_DOCTOR** table.
2. Create a new table that has **DOCTOR_ID** and **DOCTOR_NAME** columns.

With this change, the set of two tables brings this database up to 2NF.

TABLE: PATIENT_DOCTOR	
PATIENT_ID	DOCTOR_ID
1	2
2	2
3	1
4	3

Table 2-3: **PATIENT_DOCTOR** table where there are no non-key columns that do not fully depend on the composite primary key (**PATIENT_ID**, **DOCTOR_ID**)

TABLE: DOCTOR	
DOCTOR_ID	DOCTOR_NAME
1	Dr Taylor Swift
2	Dr John Reid
3	Dr Justin Bieber

Table 2-4: DOCTOR table where there are no non-key columns that do not fully depend on the primary key DOCTOR_ID

Third normal form (3NF)

For a database to be 3NF:

1. It must be 1NF and 2NF.
2. All non-key columns in each table must not be transitively dependent. i.e., All non-key columns must not be dependent on other non-key columns.

In Table 3-1 below, **DOCTOR** contains a **DOCTOR_ID** primary key. The table also has a **DOCTOR_NAME**, a **SPECIALIZATION**, and a **CONSULTING_FEE** column. The **CONSULTING_FEE** column depends on the **SPECIALIZATION** column, which is a non-key column. It does not depend on **DOCTOR_ID**, which is the primary key. **CONSULTING_FEE** is transitively dependent on **SPECIALIZATION**. Therefore, the table is not in 3NF.

TABLE: DOCTOR			
DOCTOR_ID	DOCTOR_NAME	SPECIALIZATION	CONSULTING_FEE
1	Dr Taylor Swift	Gastroenterology	USD 800
2	Dr John Reid	Cardiology	USD 1000
3	Dr Justin Bieber	Neurology	USD 1200

Table 3-1: DOCTOR table storing doctors, their specialization, and consulting fees

There is a problem with this table design. The update statement below updates the **SPECIALIZATION** column for **DOCTOR_ID** 2 to **Neurology**, but does not update the corresponding **CONSULTING_FEE**, resulting in an inconsistent consulting fee for a neurologist. This is an update anomaly.

```
UPDATE TABLE DOCTOR SET SPECIALIZATION = 'Neurology' WHERE DOCTOR_ID = 2
```

After the update, the **DOCTOR** table is in Table 3-2. In this table, even though **SPECIALIZATION** is **Neurology** in the second and third rows, the **CONSULTING_FEE** does not match.

TABLE: DOCTOR			
DOCTOR_ID	DOCTOR_NAME	SPECIALIZATION	CONSULTING_FEE
1	Dr Taylor Swift	Gastroenterology	USD 800
2	Dr John Reid	Neurology	USD 1000
3	Dr Justin Bieber	Neurology	USD 1200

Table 3-2: Update SPECIALIZATION leaves CONSULTING_FEE column inconsistent

A design change to bring the database up to 3NF would be to:

1. Remove the **CONSULTING_FEE** column from the **DOCTOR** table.
2. Create a new table **SPECIALIZATION_CONSULTING_FEE** that has **SPECIALIZATION** and **CONSULTING_FEE** columns.

With this change, the transitive dependency between **SPECIALIZATION** and **CONSULTING_FEE** is removed from the **DOCTOR** table.

With this change, the set of two tables (Table 3-3 and Table 3-4) brings this database up to 3NF.

TABLE: DOCTOR		
DOCTOR_ID	DOCTOR_NAME	SPECIALIZATION
1	Dr Taylor Swift	Gastroenterology
2	Dr John Reid	Cardiology
3	Dr Justin Bieber	Neurology

Table 3-3: DOCTOR table storing doctors and their specialization

TABLE: SPECIALIZATION_CONSULTING_FEE	
SPECIALIZATION	CONSULTING_FEE
Gastroenterology	USD 800
Cardiology	USD 1000
Neurology	USD 1200

Table 3-4: SPECIALIZATION_CONSULTING_FEE table storing specialization and consulting fees

References

1. **"Database Normalization (DBMS)."** Studytonight,
<https://www.studytonight.com/dbms/database-normalization.php>. Accessed 5 Jan. 2026.
2. Elmasri, Ramez, and Shamkant B. Navathe. **Fundamentals of Database Systems.** PDF,
<http://debracollege.dspaces.org/bitstream/123456789/619/1/Fundamentals%20of%20Database%20Systems-Elmasri%2CNavathe.pdf>. Accessed 5 Jan. 2026.